



**SIMATS**  
**SCHOOL OF ENGINEERING**  
Approved by AICTE | IET-UK Accreditation

## **CASE STUDY REPORT**

### **Digital Music Library Sorting using Merge Sort**

**Submitted By**

Name : Selvamagal M

Register Number : 192472013

**Course Code:**

CSA0603 – Design And Analysis of Algorithms

## 1. Title

Digital Music Library Sorting using Merge Sort

## 2. Description of the Real-World Problem

Music streaming apps like Spotify, Apple Music, Gaana and JioSaavn handle a huge amount of songs every single day. Spotify alone has more than hundred million tracks in its library, and new songs keep getting added almost every hour. Storing this much data is not really the hard part, the actual challenge comes when the app has to show these songs to the user in a meaningful order.

One of the most common ways users like to browse songs is by release year. A user might want to see the recent hits first, or go back and check what songs were trending in 2017 or 2018. For this kind of browsing to work properly, the backend of these apps has to sort the entire song database according to release year, and this sorting has to be fast because it is happening for millions of users at the same time.

There is another small but important detail here. Suppose two songs got released in the exact same year, like “Shape of You” and “Believer” both released in 2017. In this case the app should not shuffle them randomly every time it sorts the list. Users expect that if a song was appearing before another song earlier, it keeps appearing in that same relative position after sorting too, unless there is a reason to change it. This property is called stability in sorting, and it directly affects how consistent the app feels to a user.

This is exactly why apps like Spotify cannot use just any sorting technique. They need something that sorts fast even for very large datasets and also keeps the original order of equal elements intact. This case study looks at how the Merge Sort algorithm can solve this exact real world requirement for a digital music library.

## 3. Problem Statement

Given a list of songs where each song has a name and a release year, the task is to arrange the songs in ascending order of their release year. The catalog can contain a very large number of songs, so the sorting method used must be efficient and should not slow down as the size of data grows. Along with speed, the order of songs that share the same release year must be preserved exactly as it was in the original list, because changing this order randomly can

confuse the user browsing the app. The algorithm chosen for this problem also needs to give the same performance no matter how the input is arranged, whether partially sorted or completely random, since real music libraries keep changing every day with new releases. This case study proposes Merge Sort as a solution to this problem and studies its working, efficiency and suitability for building a sorted digital music library.

#### **4. Algorithm Used**

Merge Sort is a sorting algorithm that works on the idea of Divide and Conquer. Instead of trying to sort the whole list of songs in one shot, it keeps breaking the list into smaller and smaller pieces until each piece has only one song left in it. A list with just one element is already sorted, there is nothing left to compare there.

Once the list is broken down completely, the algorithm starts putting the pieces back together, but this time in sorted order. Two single songs are compared and merged into a sorted pair. Then two sorted pairs are compared and merged into a sorted group of four. This merging keeps happening again and again until all the small pieces come back together as one fully sorted list.

The Divide and Conquer approach basically has three parts. First is Divide, where the big problem, sorting a big list, is split into smaller sub problems, sorting smaller lists. Second is Conquer, where these smaller sub problems are solved, and for Merge Sort this happens automatically once the list becomes single elements. Third is Combine, where the solved smaller pieces are merged back together to build the final answer.

The main logic behind Merge Sort is really in the merge step. While merging two sorted lists, we look at the first element of each list and pick whichever one is smaller, then place it in the new list. This keeps repeating until one of the two lists becomes empty, after which the remaining elements of the other list are just copied over since they are already sorted. Because we always pick from the left list when both elements are equal, the relative order of equal elements never gets disturbed, and that is what gives Merge Sort its stability.

#### **5. Why Merge Sort?**

When it comes to sorting a huge and constantly growing music catalog, not every sorting algorithm fits the job. Bubble Sort and Selection Sort are simple to understand and easy to code, but they become extremely slow once the number of songs starts running into lakhs or crores, because both of them take around  $O(n^2)$  time in worst and average cases. That kind of

time complexity is fine for a college assignment with 10 or 20 elements, but it is simply not practical for something like Spotify's library.

Merge Sort on the other hand gives  $O(n \log n)$  time complexity in every case, whether the input is sorted, reverse sorted or completely random. This consistency matters a lot for a music app because the songs list keeps changing daily with new releases, and the sorting logic cannot suddenly slow down just because the input pattern changed.

Another big reason Merge Sort fits here is stability. As explained earlier, songs releasing in the same year should not get shuffled randomly. Bubble Sort and Selection Sort, depending on how they are implemented, can sometimes disturb this order, but Merge Sort naturally preserves it because of how the merge step is written.

- Stable Sorting – keeps the relative order of songs having the same release year exactly as in the original list.
- $O(n \log n)$  Time Complexity – performs efficiently in best, average and worst cases, unlike Bubble Sort and Selection Sort.
- Efficient for Huge Datasets – handles millions of songs without the performance dropping drastically.
- Consistency – gives the same predictable performance regardless of how the input list is arranged.

Because of these points, Merge Sort is a much more suitable choice than the basic sorting algorithms for building a sorting feature inside a real digital music streaming platform.

## 6. Complexity Analysis

The table below shows the time and space complexity of Merge Sort for the different cases.

| Operation        | Best Case     | Average Case  | Worst Case    |
|------------------|---------------|---------------|---------------|
| Time Complexity  | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ |
| Space Complexity | $O(n)$        | $O(n)$        | $O(n)$        |

***Explanation in simple words:***

Best Case ( $O(n \log n)$ ): Even if the song list is already sorted by release year, Merge Sort still divides the list all the way down to single elements and merges them back, so it does not get any faster than  $n \log n$ .

Average Case ( $O(n \log n)$ ): For a normal, randomly arranged song list, the algorithm splits it  $\log n$  times and merges  $n$  elements at every level, giving  $n \log n$  comparisons overall.

Worst Case ( $O(n \log n)$ ): Unlike Bubble Sort or Selection Sort which can become  $O(n^2)$  in the worst case, Merge Sort stays at  $O(n \log n)$  even for the most unfavourable arrangement of songs, which is a big advantage.

Space Complexity ( $O(n)$ ): Merge Sort needs extra memory to store the temporary left and right sub lists while merging, so it is not an in place sorting algorithm. For a large song catalog, this extra memory usage is a small tradeoff for the speed and stability it gives.

## 7. Manual Execution

This section shows how Merge Sort works step by step on paper, using the same input that was used in the program, so it becomes easy to connect the theory with the actual code output.

### *Original List (Input)*

| Song Name       | Release Year |
|-----------------|--------------|
| Shape of You    | 2017         |
| Believer        | 2017         |
| Perfect         | 2018         |
| Blinding Lights | 2020         |
| Levitating      | 2021         |
| Heat Waves      | 2020         |
| Flowers         | 2023         |
| As It Was       | 2022         |



**Step 1: Split into two halves**

Left Half:

| Song Name       | Release Year |
|-----------------|--------------|
| Shape of You    | 2017         |
| Believer        | 2017         |
| Perfect         | 2018         |
| Blinding Lights | 2020         |

Right Half:

| Song Name  | Release Year |
|------------|--------------|
| Levitating | 2021         |
| Heat Waves | 2020         |
| Flowers    | 2023         |
| As It Was  | 2022         |



**Step 2: Split again**

From Left Half → Left-Left:

| Song Name    | Release Year |
|--------------|--------------|
| Shape of You | 2017         |
| Believer     | 2017         |

From Left Half → Left-Right:

| Song Name       | Release Year |
|-----------------|--------------|
| Perfect         | 2018         |
| Blinding Lights | 2020         |

From Right Half → Right-Left:

| Song Name  | Release Year |
|------------|--------------|
| Levitating | 2021         |
| Heat Waves | 2020         |

From Right Half → Right-Right:

| Song Name | Release Year |
|-----------|--------------|
| Flowers   | 2023         |
| As It Was | 2022         |



**Step 3: Split into single element lists**

[Shape of You, 2017] [Believer, 2017] [Perfect, 2018] [Blinding Lights, 2020] [Levitating, 2021] [Heat Waves, 2020] [Flowers, 2023] [As It Was, 2022]

At this point every single list has just one song in it, so each one is already considered sorted. Now the merging back begins.



**Step 4: Merge single elements (first level of merging)**

Merge [Shape of You, 2017] and [Believer, 2017]: Compare 2017 with 2017, both years are equal so the left element is placed first (this keeps it stable).

| Song Name    | Release Year |
|--------------|--------------|
| Shape of You | 2017         |
| Believer     | 2017         |

Merge [Perfect, 2018] and [Blinding Lights, 2020]: Compare 2018 with 2020, 2018 is smaller so Perfect goes first.

| Song Name       | Release Year |
|-----------------|--------------|
| Perfect         | 2018         |
| Blinding Lights | 2020         |

Merge [Levitating, 2021] and [Heat Waves, 2020]: Compare 2021 with 2020, 2020 is smaller so Heat Waves goes first.

| Song Name  | Release Year |
|------------|--------------|
| Heat Waves | 2020         |
| Levitating | 2021         |

Merge [Flowers, 2023] and [As It Was, 2022]: Compare 2023 with 2022, 2022 is smaller so As It Was goes first.

| Song Name | Release Year |
|-----------|--------------|
| As It Was | 2022         |
| Flowers   | 2023         |



**Step 5: Merge pairs into groups of four**

Merge [Shape of You 2017, Believer 2017] and [Perfect 2018, Blinding Lights 2020]:

Compare Shape of You (2017) and Perfect (2018) → 2017 is smaller, take Shape of You.

Compare Believer (2017) and Perfect (2018) → 2017 is smaller, take Believer.

Left side is now empty, so the remaining right side elements Perfect and Blinding Lights are copied as they are.

| Song Name       | Release Year |
|-----------------|--------------|
| Shape of You    | 2017         |
| Believer        | 2017         |
| Perfect         | 2018         |
| Blinding Lights | 2020         |

Merge [Heat Waves 2020, Levitating 2021] and [As It Was 2022, Flowers 2023]:

Compare Heat Waves (2020) and As It Was (2022) → 2020 is smaller, take Heat Waves.

Compare Levitating (2021) and As It Was (2022) → 2021 is smaller, take Levitating.

Left side is now empty, so the remaining right side elements As It Was and Flowers are copied as they are.

| Song Name  | Release Year |
|------------|--------------|
| Heat Waves | 2020         |
| Levitating | 2021         |
| As It Was  | 2022         |
| Flowers    | 2023         |



**Step 6: Final Merge (whole list)**

Now the two sorted groups of four are merged together:

Compare Shape of You (2017) and Heat Waves (2020) → take Shape of You.

Compare Believer (2017) and Heat Waves (2020) → take Believer.

Compare Perfect (2018) and Heat Waves (2020) → take Perfect.

Compare Blinding Lights (2020) and Heat Waves (2020) → both years equal, left side goes first, take Blinding Lights.

Left side is now empty, so the remaining right side elements Heat Waves, Levitating, As It Was, Flowers are copied over in their sorted order.



### ***Final Sorted List (Output)***

| <b>Song Name</b> | <b>Release Year</b> |
|------------------|---------------------|
| Shape of You     | 2017                |
| Believer         | 2017                |
| Perfect          | 2018                |
| Blinding Lights  | 2020                |
| Heat Waves       | 2020                |
| Levitating       | 2021                |
| As It Was        | 2022                |
| Flowers          | 2023                |

This final list matches exactly with the output that the Python program prints, which confirms that the manual working and the code logic are in sync.

## **8. Applications**

- Sorting songs in music streaming apps like Spotify, Gaana, JioSaavn according to release year, artist name or popularity.
- Merging sorted transaction records in banking systems while keeping the daily order intact.
- External sorting of huge files that cannot fit completely into main memory, since Merge Sort works well with data stored in chunks.
- Used inside e-commerce websites to sort products by price, rating or launch date.

- Used in inventory management systems to arrange stock items by date of purchase.
- Applied in many programming languages' built in sort functions, for example Python's Timsort and Java's Arrays.sort() for objects, both borrow ideas from Merge Sort.
- Useful in sorting large log files or datasets in data analytics projects where stability of records matters.

## 9. Advantages

- Gives  $O(n \log n)$  time complexity in best, average and worst case, unlike many other simple sorting algorithms.
- It is a stable sorting algorithm, so elements with equal keys keep their original relative order.
- Performance does not depend on how the input data is arranged, it stays consistent.
- Works well on large datasets, which makes it suitable for real world applications like music apps.
- Can be used for external sorting, meaning it works even when data is too large to fit in memory at once.
- Divide and Conquer approach makes the algorithm easier to understand and implement using recursion.
- Can be parallelized since the two halves of the list can be sorted independently.
- Predictable performance makes it easier to plan system resources for large scale applications.

## 10. Limitations

- Needs extra memory space of  $O(n)$  for the temporary arrays used during merging, so it is not an in place algorithm.
- For very small lists, the overhead of splitting and merging can make it slower than simple algorithms like Insertion Sort.
- Recursive calls can lead to higher memory usage on the call stack for extremely large inputs.

- Copying elements back and forth during merging takes extra time compared to in place algorithms.
- Not the best choice for systems with very limited memory availability.
- Implementation can feel a little more complex for beginners compared to Bubble Sort or Selection Sort.

## 11. Comparison with Other Algorithms

| Basis           | Merge Sort    | Quick Sort                           | Bubble Sort |
|-----------------|---------------|--------------------------------------|-------------|
| Time Complexity | $O(n \log n)$ | $O(n \log n)$ avg,<br>$O(n^2)$ worst | $O(n^2)$    |
| Space           | $O(n)$        | $O(\log n)$                          | $O(1)$      |
| Stability       | Stable        | Not Stable                           | Stable      |
| Large Dataset   | Very Good     | Good                                 | Poor        |
| Worst Case      | $O(n \log n)$ | $O(n^2)$                             | $O(n^2)$    |

Note: Selection Sort behaves similar to Bubble Sort in terms of time complexity,  $O(n^2)$  in all cases, and it is also not a stable sort in its basic form, so it faces the same problems as Bubble Sort for this use case.

From the table it is clear that Merge Sort gives the best balance of speed, stability and consistency, which is exactly what is needed for sorting a digital music library.

## 12. Program Screenshot

```

merge sort-da.py - C:/Users/cmala/AppData/Local/Programs/Python/Python312/merge sort-d...
File Edit Format Run Options Window Help

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        # Compare release year (index 1)
        if left[i][1] <= right[j][1]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    while i < len(left):
        result.append(left[i])
        i += 1

    while j < len(right):
        result.append(right[j])
        j += 1

    return result

def merge_sort(songs):
    if len(songs) <= 1:
        return songs

    mid = len(songs) // 2
    left = merge_sort(songs[:mid])
    right = merge_sort(songs[mid:])

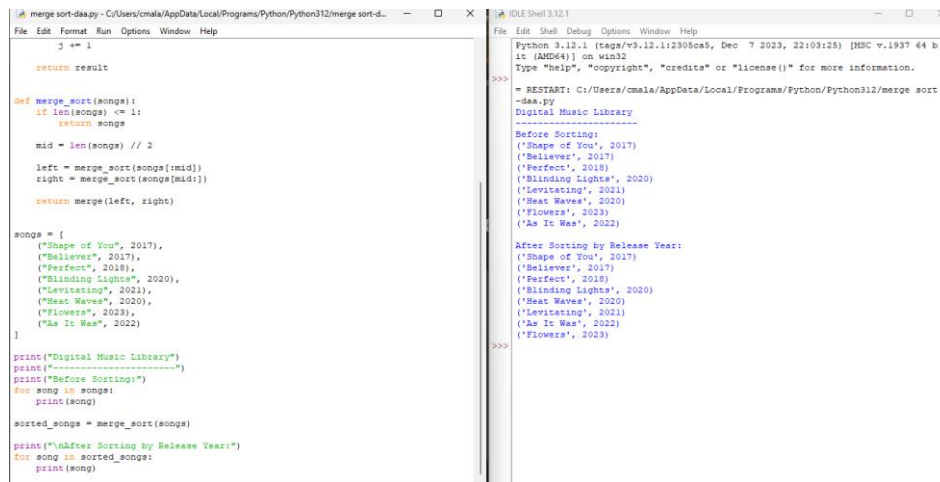
    return merge(left, right)

songs = [
    ("Shape of You", 2017),
    ("Believer", 2017),
    ("Perfect", 2018),
    ("Blinding Lights", 2020),
    ("Levitating", 2021),
    ("Heat Waves", 2020),
    ("Flowers", 2023),
    ("As It Was", 2022)
]

print("Before Sorting:")
for song in songs:
    print(song)

print("\nAfter Sorting by Release Year:")
sorted_songs = merge_sort(songs)
for song in sorted_songs:
    print(song)
  
```

Figure 1: Merge Sort Program (Part 1)



```
merge sort-da.py - C:/Users/cmala/AppData/Local/Programs/Python/Python312/merge sort-d...
File Edit Format Run Options Window Help

j += 1

return result

def merge_sort(songs):
    if len(songs) <= 1:
        return songs

    mid = len(songs) // 2
    left = merge_sort(songs[:mid])
    right = merge_sort(songs[mid:])

    return merge(left, right)

songs = [
    ('Shape of You', 2017),
    ('Believer', 2017),
    ('Perfect', 2018),
    ('Blinding Lights', 2020),
    ('Levitating', 2021),
    ('Heat Waves', 2020),
    ('Flowers', 2023),
    ('As It Was', 2022)
]

print("Digital Music Library")
print("-----")
print("Before Sorting:")
for song in songs:
    print(song)

sorted_songs = merge_sort(songs)

print("\nAfter Sorting by Release Year:")
for song in sorted_songs:
    print(song)
```

```
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [MSC v.1937 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
= RESTART: C:/Users/cmala/AppData/Local/Programs/Python/Python312/merge sort
-da.py
Digital Music Library
-----
Before Sorting:
('Shape of You', 2017)
('Believer', 2017)
('Perfect', 2018)
('Blinding Lights', 2020)
('Levitating', 2021)
('Heat Waves', 2020)
('Flowers', 2023)
('As It Was', 2022)

After Sorting by Release Year:
('Shape of You', 2017)
('Believer', 2017)
('Perfect', 2018)
('Blinding Lights', 2020)
('Heat Waves', 2020)
('Levitating', 2021)
('As It Was', 2022)
('Flowers', 2023)

>>>
```

Figure 2: Merge Sort Program (Part 2)

## 13. Output Screenshot



```
IDLE Shell 3.12.1
File Edit Shell Debug Options Window Help

Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [MSC v.1937 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
= RESTART: C:/Users/cmala/AppData/Local/Programs/Python/Python312/merge sort
-da.py
Digital Music Library
-----
Before Sorting:
('Shape of You', 2017)
('Believer', 2017)
('Perfect', 2018)
('Blinding Lights', 2020)
('Levitating', 2021)
('Heat Waves', 2020)
('Flowers', 2023)
('As It Was', 2022)

After Sorting by Release Year:
('Shape of You', 2017)
('Believer', 2017)
('Perfect', 2018)
('Blinding Lights', 2020)
('Heat Waves', 2020)
('Levitating', 2021)
('As It Was', 2022)
('Flowers', 2023)

>>>
```

Figure 3: Program Output

## 14. Conclusion

Doing this case study helped me understand Merge Sort in a much better way than just reading it from the textbook. When I was writing the Python program for sorting the songs by release year, I actually had to think about how the list breaks down and comes back together, and that made the divide and conquer idea click for me properly.

Working through the manual execution on paper also helped a lot. Seeing every single split and every merge step by step showed me exactly where the stability of the algorithm comes from, especially in cases like Blinding Lights and Heat Waves which both released in 2020. The algorithm placed Blinding Lights first because it was appearing first in the original list, and that small detail made a lot more sense once I traced it manually instead of just trusting the code output.

Comparing Merge Sort with Bubble Sort and Selection Sort also gave me a clearer picture of why real applications like Spotify cannot depend on the basic  $O(n^2)$  algorithms once the data becomes really large. The  $O(n \log n)$  time complexity of Merge Sort, along with its stability, makes it a solid choice for building features like a sorted digital music library.

Overall, this case study was a good mix of theory, coding and manual practice, and it gave me a practical example of where an algorithm taught in class is actually used in apps that we use every day.